



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 • Design & Analysis of Algorithms • Fall 2026

Workshop II — Recursion and Divide-and-Conquer

No new material. Five stations on Sessions 8–11, plus what the midterm will ask

Session 12 of 24 · Tuesday 27 October 2026 · 2:30 – 3:50 PM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 12 OF 24

Workshop II — Recursion and Divide-and-Conquer

Tuesday, 27 October 2026 · 2:30 – 3:50 PM

WHAT THIS SESSION IS FOR

- ✓ Make recurrence-writing automatic before it is needed under exam conditions.
- ✓ Practise the three regimes until the regime is visible before the arithmetic is done.
- ✓ Find out what students cannot yet do, three days before the midterm, while it can still be fixed.
- ✓ Set expectations for Friday's exam precisely, so that nobody revises the wrong thing.

How to run today. Five stations of roughly thirteen minutes, pairs at the board, the instructor circulating. Solutions are printed here for the instructor and released to students after the session. Station 5 is the midterm briefing and should not be cut — if time runs short, drop Station 3 instead.

THE FOUR IDEAS BEING CONSOLIDATED

1. Read the recurrence off the code: base case, recursive calls, glue. 2. Subtraction gives n levels; division gives $\log n$ levels. 3. Cost one level, count the levels, ask whether level costs grow, hold or shrink. 4. Merge sort = trivial divide + linear combine = $\Theta(n \log n)$, always, at $\Theta(n)$ space.

PY WRITE THE RECURRENCE FOR EACH, THEN SOLVE IT

```
def g1(n):
    if n <= 1: return 1
    return g1(n - 1) + g1(n - 1)

def g2(A):
    if len(A) <= 1: return A
    mid = len(A) // 2
    return g2(A[:mid]) + g2(A[mid:])

def g3(n):
    if n <= 1: return 1
    total = 0
    for i in range(n):
        total += i
    return g3(n // 2) + total

def g4(A, lo, hi):
    if lo >= hi: return 0
    mid = (lo + hi) // 2
    return g4(A, lo, mid) + g4(A, mid + 1, hi) + 1

def g5(n):
    if n <= 1: return 1
    return g5(n - 1) + n * n
```

SOLUTIONS

g1: $T(n) = 2T(n-1) + 1 \rightarrow \Theta(2^n)$. Two calls on $n-1$ is the fatal combination — note it computes the same value twice, an overlap disaster in miniature.

g2: $T(n) = 2T(n/2) + n \rightarrow \Theta(n \log n)$. The glue is the list concatenation, which is linear — students who call it $O(1)$ get $\Theta(n)$ and are wrong.

g3: $T(n) = T(n/2) + n \rightarrow$ level costs $n, n/2, n/4, \dots$ shrink $\rightarrow$ root dominates $\rightarrow \Theta(n)$.

g4: $T(n) = 2T(n/2) + 1 \rightarrow$ level costs double $\rightarrow$ leaves dominate $\rightarrow \Theta(n)$. This is tree traversal.

g5: $T(n) = T(n-1) + n^2 \rightarrow$ sum of squares $\rightarrow \Theta(n^3)$.

Discussion point: g2 and g4 differ only in whether the glue is linear or constant, and the answers are $\Theta(n \log n)$ and $\Theta(n)$. The glue is not a detail.

For each recurrence, students must **first** say grow / hold / shrink, *then* give Θ . Writing the regime down before computing is the whole point of the station.

Recurrence	Regime?	Θ ?
$T(n) = 2T(n/2) + n$		
$T(n) = 4T(n/2) + n$		
$T(n) = 2T(n/2) + n^3$		
$T(n) = 16T(n/4) + n^2$		
$T(n) = T(n/2) + n$		
$T(n) = 3T(n/3) + n$		

SOLUTIONS

$2T(n/2) + n$: hold $\rightarrow \Theta(n \log n)$.

$4T(n/2) + n$: grow (level costs $n, 2n, 4n$) $\rightarrow$ leaves $\rightarrow n^{\log_2 4} = \Theta(n^2)$.

$2T(n/2) + n^3$: shrink sharply $\rightarrow$ root $\rightarrow \Theta(n^3)$.

$16T(n/4) + n^2$: level costs $n^2, n^2, n^2 \rightarrow$ hold $\rightarrow \Theta(n^2 \log n)$. This one catches almost everyone; make them compute level 1 explicitly.

$T(n/2) + n$: shrink $\rightarrow$ root $\rightarrow \Theta(n)$.

$3T(n/3) + n$: hold $\rightarrow \Theta(n \log n)$.

Discussion point: rows 1, 4 and 6 are all “hold” with different a and b . The regime does not depend on a alone; it depends on a compared with b raised to the power in f .

Station 3 · Trace Merge Sort by Hand 13 min

At the board, in pairs, for $A = [38, 27, 43, 3, 9, 82, 10]$.

- 1 Draw the full division tree. How many leaves? How deep is it? Why is it not a perfect tree, and does that matter?
- 2 Write the merged result at every internal node on the way back up.
- 3 Count the total number of comparisons your trace actually performed.
- 4 Compare that count against $n \cdot \log_2 n \approx 7 \times 2.81 \approx 20$. Is your count above or below, and why?

SOLUTIONS AND TALKING POINTS

Seven leaves, depth 3 (the tree is uneven because 7 is not a power of 2 — and it does not matter, since $\lceil \log_2 7 \rceil = 3$ either way).

Merges bottom-up: [27,38], [3,43], [9,82], then [3,27,38,43] and [9,10,82], finally [3,9,10,27,38,43,82].

Comparison counts vary by trace: typically 13–16, comfortably *below* $n \log_2 n$. Merging often exhausts one side early and moves the rest with `extend`, which costs no comparisons. **The bound is an upper bound; real runs beat it.** That is worth saying out loud three days before an exam on which several students will otherwise write “it takes exactly $n \log n$ comparisons”.

Station 4 • Diagnose and Repair 13 min

PY THREE BROKEN FUNCTIONS

```
# (a) intended: merge sort. What is its actual recurrence and complexity?
def sort_a(A):
    if len(A) <= 1: return A
    mid = len(A) // 2
    return merge(sort_a(A[:mid]), sort_a(A))           # look carefully at the second call

# (b) intended: binary search. Why does it sometimes never return?
def search_b(A, x, lo, hi):
    if lo > hi: return -1
    mid = (lo + hi) // 2
    if A[mid] == x: return mid
    if A[mid] < x: return search_b(A, x, mid, hi)
    return search_b(A, x, lo, mid)

# (c) intended:  $O(n \log n)$ . What is it actually, and why?
def count_c(n):
    if n <= 1: return 1
    for i in range(n):
        for j in range(n):
            pass
    return count_c(n // 2) + count_c(n // 2)
```

SOLUTIONS

(a) The second call passes `A`, not `A[mid:]`. $T(n) = T(n/2) + T(n) + n$ — the problem never shrinks on that branch, so it never terminates. Infinite recursion, not slow code. The fix is one slice.

(b) `mid` is passed unchanged instead of `mid + 1` / `mid - 1`. When `lo` and `hi` are adjacent the range stops shrinking and the recursion spins for ever. This is the exact bug warned about in Session 1 §3.1, now in recursive form.

(c) The glue is the empty double loop: $\Theta(n^2)$. So $T(n) = 2T(n/2) + n^2 \rightarrow$ level costs shrink $\rightarrow$ root dominates $\rightarrow \Theta(n^2)$, not $\Theta(n \log n)$.

Discussion point: two of these three bugs are non-termination, and both come from getting the *size reduction* wrong rather than the logic. When a recursive function hangs, check the sizes first.

Station 5 • Midterm Briefing 15 min

What the midterm covers

Sessions 1–11. Definitions of O , Ω , Θ and witness proofs. Counting loop nests. Best / worst / average case. Writing recurrences from code.

Unrolling. Recursion trees and the three regimes.

Insertion sort, binary search, merge sort, inversion counting.

What it does not cover

Quicksort, heaps, graphs and dynamic programming — none of it has been taught yet. The Master Theorem is not examinable and will not appear. You will never be asked to recall a complexity from memory without being given the code or the recurrence it comes from.

The five question types, and how to prepare for each

Type	Looks like	How to prepare
Classify	code shown, give Θ	Station 1 and Session 5 exercises
Prove	show $f(n)$ is $O(g(n))$ from the definition	Session 4 — produce a witness
Solve	a recurrence given, find Θ	Station 2 — name the regime first
Trace	run an algorithm by hand on a small input	Station 3 — do it on paper, not mentally
Improve	slow code given, make it faster and justify	Station 4 and Session 6's Two-Sum

PRACTICAL POINTS TO STATE EXPLICITLY

One handwritten A4 sheet is permitted — the cheat sheet set as homework after Workshop I. Handwritten, one side, your own.

Partial credit is real. A correct recurrence with a wrong solution earns most of the marks. Write the recurrence down even when you cannot solve it.

Show the tree. For any recurrence question, a drawn tree with level costs labelled is itself worth marks, and it also prevents the arithmetic slip that loses them.

Bring a pen you can draw with. Half of this exam is diagrams.

The three errors most likely to appear on Friday

1. “O is the worst case, Ω is the best case” — two independent axes, Session 6.
2. Calling a dependent nest $\Theta(n)$ because it does “half the work” — half of n^2 is n^2 , Session 5.
3. Forgetting the glue when writing a recurrence, especially when the glue is a slice.

WORKSHOP II TAKEAWAY

Eleven sessions in, you can take an unfamiliar piece of code — iterative or recursive — and say how its cost grows, prove the claim, and check it against a clock. That is the entire content of the midterm, and it is the foundation for everything after it. From November onward we stop building the instruments and start using them on real algorithms.

BEFORE FRIDAY 30 OCTOBER

1. Finish any station your pair did not complete. Attempt each before reading the released solutions.
2. Extend your one-page cheat sheet to cover Sessions 8–11: the translation rules, the two knobs, the three regimes, and the merge-sort facts. One side of A4, handwritten, yours.
3. Re-do, without notes: two witness proofs from Session 4, two loop nests from Session 5, two recurrences from Session 10, and one merge-sort trace.
4. Sleep. A rested student outperforms a revised one on an exam made of diagrams and arithmetic.

Friday 30 October — Midterm Examination. Sessions 1–11. One handwritten A4 sheet permitted.

Tuesday 3 November — Session 13: Quicksort. The mirror image of merge sort — all the intelligence in the divide, nothing at all in the combine, a genuinely quadratic worst case, and the randomisation that makes it the fastest sort in practice regardless.